

50 SQL Interview Questions

Junior to Mid-Level | AI/ML & AI Engineering Roles

Concept-first. Theory-driven. Exactly what interviewers ask.

50 Questions	3 Levels	Concept-First	Code Where Needed
Junior · Mid · ML-Focus	Junior / Junior-Mid / Mid	Theory + Intuition	Only when it clarifies

Contents

1. SQL Fundamentals Junior Level	Q1 – Q15
2. Intermediate Concepts Junior-to-Mid Level	Q16 – Q30
3. Mid-Level & ML-Focused SQL Mid Level	Q31 – Q50

How to use this guide

- Read the concept first — understand the WHY before memorising the WHAT.
- Code snippets are included only when they make a concept clearer, not as boilerplate to memorise.
- The Interview Tip at the end of each question tells you exactly what to say to impress.
- Junior questions are foundational — do not skip them even if you have some experience.
- Mid-level questions lean toward ML and data engineering application — know the trade-offs, not just the definitions.

1. SQL Fundamentals

Junior Level — Q1 to Q15

Q1. What is SQL and what is it used for?

Junior

SQL (Structured Query Language) is a standard language for managing and querying relational databases. It lets you store, retrieve, update, and delete data using structured commands.

- Used in almost every data-driven application and ML pipeline.
- Comes in dialects: MySQL, PostgreSQL, SQL Server, BigQuery, Snowflake — syntax differs slightly.
- Interviewers expect you to know the four core operations: SELECT, INSERT, UPDATE, DELETE.

Interview tip: Even for ML roles, SQL is the #1 tool for data extraction, cleaning, and feature preparation.

Q2. What is the difference between DDL, DML, DCL, and TCL?

Junior

These are the four categories of SQL commands.

- DDL (Data Definition Language) — CREATE, ALTER, DROP: defines the structure of tables.
- DML (Data Manipulation Language) — SELECT, INSERT, UPDATE, DELETE: works with the data inside tables.
- DCL (Data Control Language) — GRANT, REVOKE: manages access permissions.
- TCL (Transaction Control Language) — COMMIT, ROLLBACK, SAVEPOINT: manages transactions.

Interview tip: Interviewers ask this to test your breadth. Know one example command per category.

Q3. What is the difference between WHERE and HAVING?

Junior

WHERE filters individual rows BEFORE any grouping or aggregation happens. HAVING filters the results of groups AFTER the GROUP BY clause has been applied. You cannot use aggregate functions like COUNT() or SUM() inside a WHERE clause.

- WHERE works on raw row data. HAVING works on aggregated/grouped data.
- Example: WHERE salary > 50000 filters rows; HAVING COUNT(*) > 5 filters groups.
- If there is no GROUP BY, HAVING behaves like WHERE but this is bad practice.

```
SELECT department, COUNT(*) AS total
```

```
FROM employees
```

```
WHERE status = 'active'
```

```
GROUP BY department
```

```
HAVING COUNT(*) > 10;
```

Interview tip: One of the most common junior interview questions. Saying 'HAVING is used after GROUP BY' is the key answer.

Q4. Explain the different types of JOINS in SQL.

Junior

A JOIN combines rows from two or more tables based on a related column. There are four main types.

- **INNER JOIN** — returns only rows where there is a match in BOTH tables. Non-matching rows are excluded.
- **LEFT JOIN (LEFT OUTER JOIN)** — returns ALL rows from the left table plus matched rows from the right. Unmatched right-side values come as NULL.
- **RIGHT JOIN** — opposite of LEFT JOIN. All rows from the right table are kept.
- **FULL OUTER JOIN** — returns all rows from both tables. NULLs fill in where there is no match on either side.
- **CROSS JOIN** — every row from the left paired with every row from the right (Cartesian product). Rarely used.

Interview tip: Draw a Venn diagram mentally. LEFT JOIN is by far the most common in real-world ML feature queries.

Q5. What is the order of execution of a SQL query?

Junior

SQL does NOT execute in the order you write it. The logical execution order is: FROM -> JOIN -> WHERE -> GROUP BY -> HAVING -> SELECT -> DISTINCT -> ORDER BY -> LIMIT. This is why you cannot use a SELECT alias in the WHERE clause — WHERE runs before SELECT.

- **FROM/JOIN**: tables are identified and joined first.
- **WHERE**: rows are filtered on raw data.
- **GROUP BY**: rows are grouped for aggregation.
- **HAVING**: groups are filtered.
- **SELECT**: columns and expressions are evaluated.
- **ORDER BY / LIMIT**: final sorting and row limiting.

Interview tip: Knowing this order explains many 'why doesn't this work?' bugs. Interviewers love this question.

Q6. What is the difference between UNION and UNION ALL?

Junior

Both combine the results of two SELECT statements with the same column structure. UNION removes duplicate rows (requires an internal sort/deduplicate step, so it is slower). UNION ALL keeps every row including duplicates and is faster because no deduplication is done.

- Use UNION when you genuinely need to remove cross-query duplicates.
- Use UNION ALL when duplicates are impossible, acceptable, or when performance matters.
- Both require the same number of columns and compatible data types in each SELECT.

Interview tip: In ML data pipelines combining training and staging data, always prefer UNION ALL for performance.

Q7. What is a PRIMARY KEY and a FOREIGN KEY?

Junior

A PRIMARY KEY uniquely identifies each row in a table. It cannot be NULL and must be unique. A FOREIGN KEY is a column in one table that references the PRIMARY KEY of another table, enforcing referential integrity — you cannot insert a foreign key value that does not exist in the referenced table.

- A table can have only ONE primary key (but it can span multiple columns — composite key).
- Foreign keys prevent orphaned records (e.g., an order row referencing a non-existent customer).
- In analytical/ML databases foreign key constraints are often not enforced for performance reasons.

Interview tip: Know the difference between the constraint concept and how it is implemented (or skipped) in analytical databases.

Q8. What is the difference between DELETE, TRUNCATE, and DROP?

Junior

All three remove data but in very different ways with different implications.

- DELETE removes specific rows. It can have a WHERE clause. It is logged row-by-row and can be rolled back. Slower on large tables.

- TRUNCATE removes ALL rows from a table instantly. Minimal logging, cannot filter rows, but the table structure stays intact. Much faster than DELETE for full wipes. Cannot be rolled back in most databases.

- DROP removes the entire table — both its data and its structure (schema). It is irreversible without a backup.

Interview tip: Use TRUNCATE to reset staging/temp tables in pipelines. Never confuse DROP (deletes the table itself) with DELETE (deletes rows).

Q9. What are NULL values and how do you handle them?

Junior

NULL represents the absence of a value — it is NOT zero, NOT an empty string. NULL is unknown. Any arithmetic or comparison with NULL returns NULL (not TRUE or FALSE). You must use IS NULL or IS NOT NULL to check for NULLs, not = NULL.

- NULL = NULL evaluates to NULL (unknown), not TRUE. This surprises most beginners.

- Use COALESCE(col, default_value) to replace NULLs with a fallback value.

- Use NULLIF(a, b) to return NULL when two values are equal (avoids division by zero).

- COUNT(*) includes NULLs; COUNT(column) excludes NULLs.

```
SELECT COALESCE(age, 0) AS age_filled FROM users;
```

```
SELECT revenue / NULLIF(sessions, 0) AS rev_per_session FROM metrics;
```

Interview tip: NULL handling is critical in ML — NULLs in features cause model errors. Always check with COUNT() vs COUNT(col).*

Q10. What is a subquery? What is a correlated subquery?

Junior

A subquery is a SELECT statement nested inside another SQL statement. A non-correlated subquery runs once independently of the outer query. A correlated subquery references a column from the outer query and runs once for each row in the outer query — making it much slower.

- Non-correlated: SELECT * FROM users WHERE id IN (SELECT user_id FROM orders) — inner query runs once.

- Correlated: the inner query references outer.column — it re-executes for every outer row.

- Correlated subqueries are often replaceable with JOINS or window functions for better performance.

Interview tip: Interviewers use correlated subquery questions to test whether you understand query performance.

Q11. What is the difference between CHAR and VARCHAR?

Junior

CHAR(n) stores fixed-length strings — it always uses exactly n characters, padding with spaces if the value is shorter. VARCHAR(n) stores variable-length strings up to n characters, using only as much space as needed.

- CHAR is faster for fixed-length data (e.g., country codes, status flags like 'Y'/'N').

- VARCHAR is more storage-efficient for variable-length data (names, descriptions).

- In analytical/ML databases this distinction matters less than in OLTP systems.

Interview tip: This is a common junior-level warm-up question. Lead with 'fixed vs variable length'.

Q12. What are aggregate functions? Name the main ones.

Junior

Aggregate functions perform a calculation on a set of rows and return a single value. They collapse multiple rows into one result, usually used with GROUP BY.

- COUNT(*) — total number of rows. COUNT(col) — non-NULL values only.
- SUM(col) — total of all values.
- AVG(col) — arithmetic mean of non-NULL values.
- MIN(col) / MAX(col) — smallest and largest values.
- STDDEV(col) / VARIANCE(col) — statistical spread (important for ML feature stats).

Interview tip: Know that all aggregates except COUNT() ignore NULLs. Interviewers often ask this as a follow-up to GROUP BY questions.*

Q13. What is GROUP BY and how does it work?

Junior

GROUP BY collapses rows that share the same value in the specified column(s) into a single group, allowing aggregate functions to be applied per group. Every column in the SELECT clause must either be in the GROUP BY or be inside an aggregate function.

- GROUP BY runs after WHERE and before HAVING in the execution order.
- You can group by multiple columns: GROUP BY country, department.
- A common mistake: selecting a non-aggregated column that is not in GROUP BY — this is an error in strict SQL modes.

```
SELECT department, AVG(salary) AS avg_sal
FROM employees
GROUP BY department
ORDER BY avg_sal DESC;
```

Interview tip: If asked to compute per-group statistics for ML features (e.g., average spend per user), GROUP BY is your tool.

Q14. What is an INDEX? Why is it used?

Junior

An index is a separate data structure (usually a B-tree) that the database builds on one or more columns to speed up row lookups, JOINS, and ORDER BY operations. Without an index, the database must scan every row (full table scan). With an index, it can jump directly to matching rows.

- Indexes speed up reads but slow down writes (INSERT/UPDATE/DELETE must also update the index).
- Composite indexes: covering multiple columns. The order of columns matters — the index helps queries that filter on the leading column(s).
- Too many indexes = slower writes and wasted storage. Index selectively on high-query columns.

Interview tip: In ML pipelines, always index columns used in JOIN ON and WHERE clauses for large feature tables.

Q15. What is the difference between a view and a table?

Junior

A table physically stores data on disk. A view is a saved SQL query — it stores the query definition, not the data. Every time you query a view, the underlying query re-executes. A materialized view (in some databases) physically stores the result and is refreshed periodically.

- Views simplify complex queries and enforce security (expose only certain columns).
- Regular views have no performance benefit over the raw query — they re-run every time.
- Materialized views cache results for fast access — useful for pre-computed ML features.
- You cannot INSERT into most views if they involve JOINS or aggregations.

Interview tip: Materialized views are conceptually close to a feature store's pre-computed feature tables — mention this in AI Engineering interviews.

2. Intermediate Concepts

Junior-to-Mid — Q16 to Q30

Q16. What is a CTE (Common Table Expression)?

Junior-Mid

A CTE is a temporary named result set defined at the start of a query using the WITH keyword. It exists only for the duration of that query. CTEs make complex queries readable by breaking them into named steps. They can also be recursive.

- Syntax: WITH cte_name AS (SELECT ...) SELECT * FROM cte_name.
- You can define multiple CTEs in one WITH clause, separated by commas.
- CTEs are not stored anywhere — they are just a named subquery that can be referenced multiple times.
- Recursive CTEs can traverse hierarchical data like org charts or category trees.

```
WITH active_users AS (  
  SELECT user_id FROM users WHERE status = 'active'  
)  
SELECT * FROM orders  
WHERE user_id IN (SELECT user_id FROM active_users);
```

Interview tip: CTEs are heavily used in ML feature engineering SQL to break complex multi-step transformations into readable stages.

Q17. What are window functions? How are they different from GROUP BY?

Junior-Mid

Window functions perform calculations across a set of rows related to the current row WITHOUT collapsing the rows into groups. Each row keeps its own identity in the output. GROUP BY collapses rows — you lose the individual rows. Window functions add a new computed column while preserving all original rows.

- Syntax: `function() OVER (PARTITION BY col ORDER BY col ROWS/RANGE BETWEEN ...)`
- PARTITION BY: divides rows into groups for the calculation (like GROUP BY but rows are not collapsed).
- ORDER BY inside OVER: defines the ordering within each partition.
- Common functions: `ROW_NUMBER()`, `RANK()`, `DENSE_RANK()`, `LAG()`, `LEAD()`, `SUM() OVER`, `AVG() OVER`, `NTILE()`.

```
SELECT user_id, score,  
RANK() OVER (PARTITION BY department ORDER BY score DESC) AS dept_rank  
FROM employees;
```

Interview tip: Window functions are the most important mid-level SQL topic. Expect at least one window function question in every data/ML interview.

Q18. Explain ROW_NUMBER, RANK, and DENSE_RANK. How do they differ?

Junior-Mid

All three assign a number to each row within a partition ordered by a specified column, but they handle ties differently.

- `ROW_NUMBER()` — always assigns unique sequential numbers (1,2,3,4...) even for tied values. Ties get arbitrary ordering.
- `RANK()` — tied rows get the same rank, but the next rank SKIPS. Ties at position 2 both get rank 2, next row gets rank 4.
- `DENSE_RANK()` — tied rows get the same rank, but the next rank does NOT skip. Ties at position 2 both get rank 2, next row gets rank 3.
- Example: scores 100,100,90,80 -> `ROW_NUMBER`: 1,2,3,4 | `RANK`: 1,1,3,4 | `DENSE_RANK`: 1,1,2,3.

Interview tip: This is one of the most commonly asked mid-level SQL questions. Memorise the tie-breaking behaviour for each.

Q19. What are LAG and LEAD functions?

Junior-Mid

LAG(column, n) returns the value of column from the row n positions BEFORE the current row within the partition. LEAD(column, n) returns the value n positions AFTER the current row. Both are window functions and are invaluable for time-series comparisons.

- Default offset is 1 if not specified: LAG(col) gives the immediately preceding row.
- A third optional argument provides a default value if the offset goes out of bounds (e.g., first row has no previous row).
- Used for: computing day-over-day changes, creating lag features for ML, detecting transitions in state.

```
SELECT date, revenue,  
LAG(revenue, 1, 0) OVER (ORDER BY date) AS prev_day_revenue,  
revenue - LAG(revenue,1,0) OVER (ORDER BY date) AS daily_change  
FROM daily_sales;
```

Interview tip: LAG is the primary tool for creating time-series lag features in SQL — a must-know for any ML Engineering role.

Q20. What is the difference between INNER JOIN and LEFT JOIN? When would you use each?

Junior-Mid

INNER JOIN returns only rows with a match in both tables — unmatched rows are discarded. LEFT JOIN returns ALL rows from the left table regardless of whether there is a match; unmatched columns from the right table appear as NULL. Use INNER JOIN when you only care about complete data. Use LEFT JOIN when the left table is your primary dataset and the right table is optional context.

- LEFT JOIN is safe when joining features to a user table — users without features keep their row (features are NULL).
- INNER JOIN can silently drop rows, which is dangerous in ML training data — you may lose labeled examples.
- Checking the row count before and after a join reveals silent data loss from unexpected INNER JOIN behavior.

Interview tip: In ML pipelines, prefer LEFT JOIN over INNER JOIN by default to avoid accidentally dropping labeled rows.

Q21. What is normalisation? What are 1NF, 2NF, 3NF?

Junior-Mid

Normalisation is the process of organising a database to reduce data redundancy and improve data integrity by dividing data into related tables.

- 1NF (First Normal Form): every column contains atomic (single, indivisible) values. No repeating groups or arrays in a cell.
- 2NF (Second Normal Form): 1NF satisfied AND every non-key column depends on the WHOLE primary key (eliminates partial dependencies). Relevant when the primary key is composite.
- 3NF (Third Normal Form): 2NF satisfied AND no non-key column depends on another non-key column (eliminates transitive dependencies).
- OLAP/ML databases are often intentionally denormalized for query speed — joins are expensive at scale.

Interview tip: Interviewers test this to see if you understand database design. Know the definitions and one example violation for each.

Q22. What is a self-join and when is it useful?

Junior-Mid

A self-join joins a table to itself. You use table aliases to distinguish the two 'copies'. It is useful for comparing rows within the same table — hierarchical relationships (employee-manager), finding pairs, or detecting duplicates.

- Classic use case: find employees who earn more than their manager.
- The key technique: alias the same table twice (e.g., e for employee, m for manager).
- Also useful in ML: comparing a model's prediction on the same entity at different time points.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

Interview tip: Self-join questions are a classic interview problem. Always use clear aliases — the logic is easy to confuse without them.

Q23. What is the difference between TRUNCATE and DELETE? Can you roll back TRUNCATE?

Junior-Mid

DELETE is a DML operation — it removes rows one by one, is fully logged, can use WHERE to be selective, and can be rolled back within a transaction. TRUNCATE is a DDL operation — it removes all rows by deallocating the data pages, is minimally logged, cannot filter rows, and in most databases (SQL Server, Oracle) CANNOT be rolled back. In PostgreSQL, TRUNCATE CAN be rolled back.

- TRUNCATE resets identity/auto-increment counters; DELETE does not.
- TRUNCATE is much faster on large tables because it does not log individual row deletions.
- In pipeline design: TRUNCATE staging tables between runs; use DELETE only when you need to remove specific rows.

Interview tip: The rollback behaviour of TRUNCATE varies by database — mention this in interviews to show awareness of database differences.

Q24. What are ACID properties in a database?

Junior-Mid

ACID describes the four guarantees that a reliable database transaction must provide.

- Atomicity — a transaction is all-or-nothing. If any step fails, the entire transaction is rolled back.
- Consistency — the database moves from one valid state to another. No partial writes that violate constraints.
- Isolation — concurrent transactions do not interfere with each other. Results are as if transactions ran serially.
- Durability — once a transaction is committed, it persists even if the system crashes (written to disk).

Interview tip: ACID is fundamental for ML pipeline reliability — if a feature write fails mid-way, atomicity ensures no partial data is committed.

Q25. What is a composite key? When would you use one?

Junior-Mid

A composite key is a primary key made up of two or more columns. No single column uniquely identifies a row, but the combination of columns does. It is used when a natural unique identifier requires multiple fields.

- Example: an `order_items` table where neither `order_id` alone nor `product_id` alone is unique, but the pair (`order_id`, `product_id`) is.
- Composite keys are common in many-to-many relationship junction tables.
- In feature stores, (`entity_id`, `feature_name`, `as_of_timestamp`) often forms a composite key.

Interview tip: Mention composite keys when discussing feature store schema design — it shows you think about data modeling.

Q26. Explain the difference between OLTP and OLAP.

Junior-Mid

OLTP (Online Transaction Processing) databases are optimised for many small, fast read/write operations on individual records. They are normalised, row-oriented, and power day-to-day applications. OLAP (Online Analytical Processing) databases are optimised for complex analytical queries scanning millions of rows. They are often denormalised, column-oriented, and power reporting and ML training data workloads.

- OLTP examples: MySQL, PostgreSQL used by web apps.
- OLAP examples: BigQuery, Redshift, Snowflake, ClickHouse.
- ML training data almost always comes from OLAP systems or data lakes.
- Column-store (OLAP) is faster for aggregate queries because only relevant columns are read from disk.

Interview tip: For AI Engineering roles, understanding why ML training data lives in OLAP systems is important for system design discussions.

Q27. What are the different types of indexes? (B-tree, Hash, Bitmap, Composite)

Junior-Mid

Different index types suit different query patterns.

- B-tree index (default): sorted tree structure. Supports equality, range queries (`<`, `>`, `BETWEEN`), and `ORDER BY`. Works for most use cases.
- Hash index: maps values to hash buckets. Only supports exact equality lookups (`=`). Faster than B-tree for equality but useless for range queries.
- Bitmap index: stores a bit per distinct value per row. Very efficient for low-cardinality columns (e.g., `gender`, `status`). Common in data warehouses.
- Composite (multi-column) index: index on multiple columns. The `ORDER` of columns matters — the leftmost column must be in the query filter for the index to be used.

Interview tip: B-tree is the default everywhere. Mention bitmap indexes when discussing data warehouse query optimisation for ML analytics.

Q28. What is a stored procedure? How is it different from a function?

Junior-Mid

A stored procedure is a saved block of SQL code with optional parameters that can be executed by name. A function also contains logic but **MUST** return a value and can be used directly inside a SELECT query. Procedures can have side effects (INSERT, UPDATE) while functions in most databases should be side-effect-free.

- Procedures: called with EXECUTE or CALL. Can modify data. Cannot be used inside SELECT.
- Functions: called inside expressions like SELECT my_func(col). Must return a value.
- Both are reusable, pre-compiled (often), and help encapsulate business logic in the database layer.

Interview tip: In modern ML pipelines, stored procedures are less common — logic lives in Python orchestrators (Airflow, dbt) — but the concept is still tested.

Q29. What is a trigger in SQL?

Junior-Mid

A trigger is a stored procedure that automatically executes in response to a specific event on a table: BEFORE or AFTER an INSERT, UPDATE, or DELETE. Triggers enforce business rules, maintain audit logs, or automatically update related tables.

- BEFORE trigger: runs before the data change — can modify or validate the incoming data.
- AFTER trigger: runs after the data change — used for logging or cascading updates.
- Triggers are hidden logic — they execute automatically without being called explicitly, which can make debugging hard.
- In ML systems, triggers are sometimes used to log model prediction inserts into audit tables.

Interview tip: Understand the concept and trade-offs. Over-use of triggers creates hidden complexity — mention this in design discussions.

Q30. What is query optimisation? What does EXPLAIN do?

Junior-Mid

Query optimisation is the process of choosing the most efficient execution plan for a SQL query. The database's query planner/optimiser does this automatically, but you can influence it with indexes, query rewrites, and hints. EXPLAIN shows the execution plan the optimiser chose without running the query. EXPLAIN ANALYZE runs the query and shows actual timing.

- Look for Seq Scan (full table scan) — a missing index.
- Look for Nested Loop on large tables — may need an index or a different join strategy.
- Look for high 'cost' estimates — the planner's estimated work units.
- Hash Join is generally fast for large tables. Index Scan is fast for small result sets.

```
EXPLAIN ANALYZE
```

```
SELECT * FROM training_data WHERE label = 1;
```

Interview tip: Saying 'I always EXPLAIN queries before running them on production tables' is a strong signal for any data/ML engineering role.

3. Mid-Level & ML-Focused SQL

Mid Level — Q31 to Q50

Q31. What is a recursive CTE and when is it used?

Mid

A recursive CTE is a CTE that references itself. It has two parts joined by UNION ALL: a base case (the starting rows) and a recursive case (rows derived from the previous iteration). It continues until no new rows are produced. Used to traverse hierarchical or graph-structured data.

- Classic uses: org chart traversal (find all reports under a manager), folder/category hierarchies, bill-of-materials explosions.
- In AI systems: traversing knowledge graphs, feature dependency trees, category taxonomies.
- Always include a termination condition or a depth limit to prevent infinite loops.
- Not all databases support recursive CTEs — BigQuery, PostgreSQL, SQL Server do; MySQL 8+ does.

Interview tip: Recursive CTEs come up in system design interviews when you discuss how to store and query hierarchical data at scale.

Q32. What is the difference between a correlated subquery and a window function? Which is better?

Mid

A correlated subquery runs once per outer row — $O(n)$ executions, making it slow on large datasets. A window function is computed in a single pass over the data by the query engine, making it far more efficient. Any correlated subquery that computes a row-level aggregate (running total, rank, previous value) can usually be rewritten as a window function.

- Correlated subquery to get each employee's department average: slow — runs one subquery per employee.
- Window `AVG() OVER (PARTITION BY dept)`: fast — single pass.
- Rule of thumb: if a subquery references the outer table, rewrite it as a window function or JOIN.

Interview tip: Interviewers ask this to test whether you can write performant SQL. Always prefer window functions over correlated subqueries.

Q33. Explain partitioning in databases. What types exist?

Mid

Partitioning splits a large table into smaller physical segments (partitions) while maintaining a single logical table. Queries that filter on the partition key only scan relevant partitions — called partition pruning — dramatically reducing I/O.

- Range partitioning: rows go into partitions based on a value range. Most common: partition by date (e.g., one partition per month). Standard for ML training data tables.
- List partitioning: rows go into partitions based on an explicit list of values (e.g., partition by country).
- Hash partitioning: rows distributed across N partitions by `hash(key) % N`. Ensures even distribution.
- In cloud warehouses (BigQuery, Snowflake, Redshift): partition by date is the standard cost-saving technique.

Interview tip: Always mention partitioning by date when discussing how to store large ML training datasets in cloud warehouses.

Q34. What is a materialized view and how is it different from a regular view?

Mid

A regular view is just a saved query — it re-executes every time you query it. A materialized view physically stores the query result on disk. It is stale until refreshed. Querying a materialized view is instant because the data is pre-computed. It is ideal for expensive aggregations that are queried frequently.

- Regular view: always up to date, but slow for complex queries.
- Materialized view: fast, but data may be stale between refreshes.
- Refresh strategies: ON COMMIT (immediate, expensive), ON DEMAND (manual or scheduled), INCREMENTAL (only refresh changed rows — supported in some databases).
- Feature stores conceptually work like scheduled materialized views — pre-computing features for low-latency serving.

Interview tip: The materialized view concept is directly analogous to how batch feature stores work. Mentioning this connection impresses interviewers.

Q35. What are slowly changing dimensions (SCD)?

Mid

A slowly changing dimension (SCD) is a dimension table (e.g., user attributes, product details) where values change slowly over time. The key challenge is deciding how to handle historical changes for point-in-time accuracy.

- SCD Type 1: simply overwrite the old value. No history kept. Simple but loses historical accuracy.
- SCD Type 2: add a new row for each change with `valid_from`, `valid_to`, and `is_current` columns. Full history is preserved. This is the standard for ML features — you can reconstruct the value at any past point in time.
- SCD Type 3: add a 'previous_value' column. Limited history (only one version back).
- Point-in-time correctness: in ML training, you must use the feature value as it existed at prediction time, not today's value. SCD Type 2 enables this.

Interview tip: SCD Type 2 and point-in-time correctness are critical ML Engineering concepts. Label leakage often occurs from ignoring this.

Q36. What is a star schema versus a snowflake schema?

Mid

Both are data warehouse schema designs. A star schema has a central fact table (events, transactions) surrounded by denormalized dimension tables. It is simple and fast — queries need few joins. A snowflake schema normalizes the dimension tables further into sub-dimensions, reducing redundancy but requiring more joins and being slower to query.

- Fact table: stores measurable events (sales, clicks, predictions) with foreign keys to dimensions.
- Dimension table: descriptive context (product details, user profiles, date attributes).
- Star schema is preferred for ML feature extraction — fewer joins = faster training data queries.
- Snowflake schema is preferred when storage efficiency and data integrity matter more than query speed.

Interview tip: When asked about ML training data warehouse design, recommend a star schema and explain why fewer joins matter at scale.

Q37. What is the N+1 query problem?

Mid

The N+1 problem occurs when application code runs 1 query to fetch N parent records, then N additional queries to fetch related data for each record — a total of N+1 database round trips. This destroys performance at scale.

- Example: fetch 1000 users, then for each user make a separate query to get their features. That is 1001 queries.
- Fix: use a single JOIN or IN clause to fetch all related data in one query.
- In ML feature serving: if your feature retrieval code makes one DB call per entity, it will not scale. Batch retrieval is essential.
- ORMs (like SQLAlchemy) are a common source of N+1 — always check generated SQL in ML pipelines.

Interview tip: This comes up in ML Engineering system design. Batch feature retrieval (fetch all at once) vs per-entity lookups is a real architectural decision.

Q38. How does an UPSERT work? What is ON CONFLICT?

Mid

An UPSERT inserts a row if it does not exist, or updates it if it does — as a single atomic operation. In PostgreSQL, this is done with INSERT ... ON CONFLICT DO UPDATE. In MySQL, with INSERT ... ON DUPLICATE KEY UPDATE. In SQL Server and BigQuery, with the MERGE statement.

- UPSERT is essential for incremental feature table updates — only recompute and write changed entities.
- Without UPSERT, you would need separate SELECT + INSERT/UPDATE logic with race conditions.
- ON CONFLICT (column) specifies which uniqueness constraint triggers the conflict detection.
- EXCLUDED refers to the row that was rejected — use it to reference the incoming values in the UPDATE clause.

```
INSERT INTO user_features (user_id, score, updated_at)
VALUES (42, 0.91, NOW())
ON CONFLICT (user_id)
DO UPDATE SET score = EXCLUDED.score,
updated_at = EXCLUDED.updated_at;
```

Interview tip: Upserts are the standard pattern for incrementally updating ML feature tables without reprocessing everything.

Q39. What is query pushdown and why does it matter in ML pipelines?

Mid

Query pushdown means the compute engine (e.g., Spark, a BI tool, or an API) sends filter and aggregation operations down to the storage layer or source database to execute there — before data is transferred. This minimises data movement.

- Without pushdown: pull the entire 10TB table into Spark, THEN filter. Extremely slow and expensive.
- With pushdown: filter at the source, transfer only the matching rows. 100x faster and cheaper.
- When writing SQL for ML data extraction, always apply WHERE, GROUP BY, and LIMIT at the source query level.
- Connector libraries (JDBC, BigQuery connector for Spark) support predicate pushdown — always enable it.

Interview tip: This is a practical AI Engineering concept. Saying 'I always filter at the source to minimise data movement' signals production experience.

Q40. What is the difference between RANK() and DENSE_RANK() in a real use case?

Mid

Use RANK() when gaps in ranking are meaningful (e.g., 'you ranked 5th out of 100' — positions 3 and 4 were tied at rank 3, so you are genuinely 5th). Use DENSE_RANK() when you want tier groupings without gaps (e.g., 'top 3 performance tiers' — you want exactly 3 distinct tier numbers regardless of ties).

- RANK() example: leaderboard where position matters — '3rd place' is meaningful.
- DENSE_RANK() example: bucketing users into performance quintiles — you want exactly 5 groups.
- For selecting the top-N distinct values per group, DENSE_RANK is usually safer.
- ROW_NUMBER() when you need a guaranteed unique sequential number regardless of ties.

Interview tip: Interviewers often ask 'which would you use for top-N per group?' — answer DENSE_RANK and explain why.

Q41. How do you detect duplicate rows in a large table?

Mid

Use GROUP BY on the columns that define uniqueness, then filter with HAVING COUNT(*) > 1 to find groups that appear more than once.

- Group by all columns you want to be unique (e.g., user_id + event_date + event_type).
- HAVING COUNT(*) > 1 reveals duplicates.
- To keep only one copy: use ROW_NUMBER() OVER (PARTITION BY duplicate_key ORDER BY created_at DESC) and keep rn = 1.
- Duplicates in ML training data inflate the weight of certain examples and bias the model.

```
SELECT user_id, event_date, COUNT(*) AS cnt
FROM events
GROUP BY user_id, event_date
HAVING COUNT(*) > 1;
```

Interview tip: Deduplication is one of the first data quality checks in any ML pipeline. Know both how to detect and how to remove duplicates.

Q42. How do you compute a rolling (moving) average in SQL?

Mid

Use AVG() as a window function with a ROWS BETWEEN frame specification. The frame defines how many rows before and after the current row to include in the average.

- ROWS BETWEEN 6 PRECEDING AND CURRENT ROW gives a 7-day trailing average.
- ROWS BETWEEN 3 PRECEDING AND 3 FOLLOWING gives a centered 7-day average.
- ROWS (physical row count) vs RANGE (value-based) — use ROWS for time-series to avoid issues with tied dates.
- Rolling averages are a fundamental time-series feature for churn, demand, and anomaly detection models.

```
SELECT date, value,  
AVG(value) OVER (  
ORDER BY date  
ROWS BETWEEN 6 PRECEDING AND CURRENT ROW  
) AS rolling_7d_avg  
FROM daily_metrics;
```

Interview tip: Rolling averages in SQL avoid the need to pull raw time-series into Python just to compute a simple moving average.

Q43. What is a pivot table in SQL? How do you implement it?

Mid

Pivoting converts rows into columns — it reshapes data from a long format (one row per attribute) to a wide format (one column per attribute). Most databases implement this with conditional aggregation using CASE WHEN inside SUM() or MAX().

- Identify the unique values that will become column names.
- Use CASE WHEN feature_name = 'X' THEN value END inside an aggregate function.
- Group by the entity identifier (e.g., user_id).
- Some databases (SQL Server, BigQuery) have a native PIVOT syntax, but conditional aggregation is more portable.

```
SELECT user_id,  
MAX(CASE WHEN feat='age' THEN val END) AS age,  
MAX(CASE WHEN feat='income' THEN val END) AS income  
FROM features_long  
GROUP BY user_id;
```

Interview tip: Pivoting is essential when extracting features from a feature store stored in a long/EAV format into a wide training matrix.

Q44. What is the difference between ROWS BETWEEN and RANGE BETWEEN in window functions?

Mid

Both define the window frame for window function calculations, but they interpret the boundary differently. ROWS BETWEEN counts physical rows — it is precise and predictable. RANGE BETWEEN counts rows within a value range — it includes all rows with the same ORDER BY value, which can cause unexpected behavior with ties.

- ROWS BETWEEN 2 PRECEDING AND CURRENT ROW: always includes exactly the 3 most recent physical rows.
- RANGE BETWEEN 2 PRECEDING AND CURRENT ROW on an integer col: includes all rows where value $\geq$ (current - 2). Could be many rows if there are ties.
- For time-series rolling calculations, always use ROWS BETWEEN — ties in dates will produce inconsistent results with RANGE.

Interview tip: This is an advanced detail that separates mid-level from junior candidates. Defaulting to ROWS BETWEEN is the safe, correct choice.

Q45. What is data normalisation vs denormalisation in the context of ML?

Mid

Database normalisation (1NF/2NF/3NF) reduces redundancy by splitting data into related tables. Denormalisation intentionally introduces redundancy by merging tables — trading storage for query speed. For ML training data, denormalised wide tables (feature matrices) are preferred because joining many tables at training time is slow.

- Normalised schema: good for OLTP (consistent writes, no redundancy).
- Denormalised schema: good for OLAP and ML (fast reads, pre-joined data).
- ETL/ELT pipelines typically denormalise source data into flat training tables.
- dbt (data build tool) is widely used to orchestrate SQL-based denormalisation for ML feature preparation.

Interview tip: Interviewers for ML Engineering roles expect you to explain WHY you would denormalise data for model training.

Q46. How do you implement a train/test split deterministically in SQL?

Mid

Use a hash function or modulo arithmetic on the primary key to assign each row to a split. This is deterministic — the same row always gets the same split assignment across runs, ensuring reproducibility.

- `MOD(user_id, 10) < 8` assigns approximately 80% to training, 10% to validation, 10% to test.
- For truly random but stable splitting, use a hash: `MOD(ABS(FARM_FINGERPRINT(CAST(id AS STRING))), 10)` in BigQuery.
- Never use `ORDER BY RAND()` for splitting — it is non-deterministic and changes every run.
- Time-series data: always split by time (train on past, test on future) — random splits cause label leakage.

```
SELECT *, CASE
WHEN MOD(user_id, 10) < 8 THEN 'train'
WHEN MOD(user_id, 10) = 8 THEN 'val'
ELSE 'test'
END AS split
FROM labeled_data;
```

Interview tip: Deterministic splitting is a data science best practice that shows maturity. Mention time-based splitting for time-series data.

Q47. How do you compute class imbalance and basic label statistics in SQL?

Mid

Use `COUNT(*)` with conditional aggregation (`CASE WHEN`) to compute per-class counts and ratios. This is the SQL equivalent of `value_counts()` in pandas.

- Count positives, count negatives, compute the positive rate.
- Also compute: total rows, null label count, distribution across classes for multi-class problems.
- Imbalance ratio informs whether to use class weights, oversampling (SMOTE), or threshold adjustment in the model.
- Target leakage check: if positive rate is suspiciously high (>50% for rare events), investigate if any future information leaked into features.

```
SELECT
SUM(CASE WHEN label=1 THEN 1 ELSE 0 END) AS positives,
SUM(CASE WHEN label=0 THEN 1 ELSE 0 END) AS negatives,
ROUND(AVG(label), 4) AS positive_rate
FROM training_labels;
```

Interview tip: Always compute label statistics before training. A positive rate of 0.001 means the model will predict all zeros unless you address imbalance.

Q48. What is a confusion matrix? Can you compute one in SQL?

Mid

A confusion matrix summarises the performance of a classification model by counting four outcomes: True Positives (predicted positive, actually positive), False Positives (predicted positive, actually negative), True Negatives, and False Negatives. In SQL, this is computed with conditional aggregation.

- TP: model said YES, reality YES.
- FP: model said YES, reality NO (Type I error).
- FN: model said NO, reality YES (Type II error — often more costly).
- TN: model said NO, reality NO.
- Precision = $TP / (TP + FP)$. Recall = $TP / (TP + FN)$. F1 = harmonic mean of precision and recall.

```
SELECT
SUM(CASE WHEN actual=1 AND pred=1 THEN 1 ELSE 0 END) AS TP,
SUM(CASE WHEN actual=0 AND pred=1 THEN 1 ELSE 0 END) AS FP,
SUM(CASE WHEN actual=1 AND pred=0 THEN 1 ELSE 0 END) AS FN,
SUM(CASE WHEN actual=0 AND pred=0 THEN 1 ELSE 0 END) AS TN
FROM model_predictions;
```

Interview tip: Being able to compute model evaluation metrics directly in SQL (without Python) is a strong differentiator for AI Engineering roles.

Q49. What are common SQL anti-patterns to avoid?

Mid

Anti-patterns are common but problematic approaches that harm performance, correctness, or maintainability.

- SELECT * in production code: pulls unnecessary columns, breaks if schema changes, prevents index-only scans.
- Implicit conversions: comparing a VARCHAR column with an integer forces a type cast and prevents index use.
- Functions on indexed columns in WHERE: WHERE YEAR(date) = 2024 prevents the index on 'date' from being used.
- Leading wildcard LIKE: WHERE name LIKE '%smith' cannot use a B-tree index.
- Correlated subqueries when a window function or JOIN would work — $O(n)$ executions.
- NOT IN with a subquery that can return NULLs: if the subquery returns any NULL, the entire NOT IN returns no rows. Use NOT EXISTS instead.
- Overusing DISTINCT as a fix: DISTINCT often masks a JOIN that produces duplicates — fix the root cause.

Interview tip: Listing 3-4 anti-patterns with explanations in an interview shows real production experience, not just textbook knowledge.

Q50. How would you design a SQL schema for storing ML model features (a simple feature store)?

Mid

A feature store needs to: store feature values per entity, support point-in-time lookups (what was the value at time T?), and allow efficient batch retrieval for training and low-latency lookup for serving.

- **entities** table: `entity_id` (PK), `entity_type` (user/item/session).
- **feature_definitions** table: `feature_name` (PK), `data_type`, `description`, `owner`, `created_at`.
- **feature_values** table: (`entity_id`, `feature_name`, `value`, `as_of_timestamp`) — composite PK. Partitioned by `as_of_timestamp` for efficient time-range queries.
- **model_feature_registry**: (`model_id`, `feature_name`, `version`) — tracks which features each model version uses.
- **Point-in-time join**: to get features as of training time, filter `WHERE as_of_timestamp <= label_timestamp` and take the latest value per (entity, feature) pair.
- **Serving**: index on (`entity_id`, `feature_name`) for fast real-time lookup.

Interview tip: This is an open-ended system design question. Mentioning point-in-time correctness, partitioning, and the training vs serving distinction will set you apart.

Quick Revision Checklist

Junior

- SQL execution order: FROM > WHERE > GROUP BY > HAVING > SELECT > ORDER BY
- WHERE vs HAVING — WHERE filters rows, HAVING filters groups
- INNER vs LEFT JOIN — LEFT JOIN keeps all left rows, NULLs for no match
- NULL rules — NULL = NULL is NULL, not TRUE. Use IS NULL to check.
- DELETE (selective, rollback-able) vs TRUNCATE (all rows, fast) vs DROP (table gone)
- Aggregate functions ignore NULLs except COUNT(*)
- Index = faster reads, slower writes. Index JOIN and WHERE columns.

Junior-Mid

- CTE with WITH keyword — readable named subquery, not stored anywhere
- Window functions preserve rows; GROUP BY collapses them
- ROW_NUMBER (unique) vs RANK (gaps on ties) vs DENSE_RANK (no gaps on ties)
- LAG / LEAD — access previous / next row value within a partition
- Correlated subquery = O(n) — replace with window function or JOIN
- ACID: Atomicity, Consistency, Isolation, Durability
- OLTP (fast writes, normalised) vs OLAP (fast reads, denormalised)

Mid

- Partitioning by date = #1 cost saver in cloud warehouses for ML data
- SCD Type 2 — new row per change with valid_from/valid_to for point-in-time accuracy
- Materialised views = pre-computed, physically stored query results
- Upsert = INSERT + UPDATE in one atomic step. Use ON CONFLICT in PostgreSQL.
- Train/test split: use MOD(id, 10) for deterministic, reproducible splits
- Confusion matrix in SQL: conditional aggregation with CASE WHEN
- Feature store schema: entity_id + feature_name + value + as_of_timestamp

You are prepared. Go get that role.